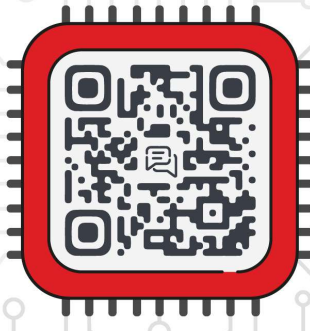




8

7

1200 sp

نظري

محتوى مجاني غير مخصص للبيع التجاري

RB Informatcs; 11/05/2026

كلية الهندسة المعلوماتية

السنة الرابعة

تتمة البنيان
المؤسساتي

د. عمار جوخدار



برمجة تفرعية

تعرفنا في المحاضرة السابقة على مجموعة من Design patterns مثل:

(SOA , AOP , Aspect , Message , FACADE , DAO , ORM , 3 Tier , MVC)

ما هو الـ SOA ؟

هي البنية الخدمية (Service Oriented Architecture) أو تسمى MicroServices , و تعني أن الخدمات مستقلة عن بعضها البعض و عندها أستطيع أن أرفعها على سيرفر (Deploy) أو نشرها لوحدها ، أي يمكن تعديلها دون الحاجة إلى تعديل جميع أجزاء الكود.



ما معنى Bottleneck ؟

يجب تمييز نوعين للـ Bottleneck و هما منطقية و فيزيائية:

■ الـ Bottleneck المنطقية:

نقطة منطقية تمر من خلالها جميع الأكواد مثل الـ FACADE و وجودها لا يسبب تباطؤ النظام.

■ الـ Bottleneck الفيزيائية:

الموارد المشتركة التي يتم طلبها من العديد من المصادر و التي تكون محدودة أو غير كافية لأداء العمل المطلوب، مما يسبب تباطؤ حقيقي في النظام مثل وجود جهاز واحد يؤدي مهمة معينة لكافة الطلبات.

سؤال مهم : هل تمتلك الـ FACADE موارد خاصة بها؟

لا، لا تستهلك أية موارد وإنما تستخدم موارد الـ Thread الذي قام باستدعائها، فهي بحد ذاتها لا تعتبر Enterprise Object، ولها دور في تطبيق السمات القبلية (Before) و السمات البعدية (After).

من أهم أدوار الـ FACADE هو إيجاد الخدمة (في حال نقلها على سيرفر آخر)

في حال كانت الخدمة المطلوبة موجودة على عدة سيرفرات كيف تختار الـ FACADE السيرفر المناسب؟

تعتمد الـ FACADE على ثلاث استراتيجيات في اختيار الخدمة المناسبة الموجودة على عدة سيرفرات :

1. أقل حمل على السيرفر (السيرفر الذي يكون جاهز للطلبات).

2. عشوائياً.

3. بالترتيب (الأول فالثاني فالثالث ...).

و كل هذه الاستراتيجيات للـ Load Balancing لا تضمن توزيع العبء بشكل كامل.

■ توضيح لما سبق :

FACADE → Aspect → Business Service → Aspect

1. FACADE :

وصل لعندها طلب معين يريد خدمة معينة

2. Aspect :

تستدعي الـ Before المناسب للخدمة (ممکن يكونوا أكثر من Aspect و ليس شرط واحد فقط).

3. Business Service :

تستدعي الخدمة المناسبة للطلب

4. Aspect :

بعد تنفيذ الخدمة تستدعي الـ After ، في حال النجاح للخدمة أو OnExcep في حال فشل تنفيذ الخدمة

يمكن تحقيق الـ Before بـ Start و الـ After بـ Commit و الـ OnExcep بـ RollBack

ملاحظة:

الـ Security تمتلك Start فقط لأنها إما تسمح للمستخدم بالعبور أو لا تسمح

أي غرض يحقق Before, After OnExcep يمكن اعتباره أنه Aspect ، أي جميع المتطلبات الغير وظيفة هي عبارة عن Aspect .

سؤال مهم : لماذا يكون كود الـ Aspect مشترك لدى كل توابع الخدمات؟

لأن جميع التوابع تحوي نفس التوقيع لذلك من المهم توحيد التوقيع.

أكواد البرمجة الموجهة بالجوانب

AOP — Aspect-Oriented Programming

```
Interface Aspect {
    Msg before(Msg input);
    Msg after(Msg result);
    Msg onException(Exception ex);
}
```

- تُعرّف هذه الشيفرة واجهة (interface) باسم Aspect تُمثّل العقد الأساسي لأي جانب (Aspect) في النظام. تحتوي على ثلاث دوالٍ يجب أن يُنفّذها كل صنف يطبّق هذه الواجهة: before() تُنفّذ قبل تنفيذ العملية الأساسية، وafter() تُنفّذ بعد نجاحها، و onException() تُنفّذ في حال حدوث استثناء (Exception) أثناء التنفيذ.

```
Class Facade {
    static List aspects;
    static void register(Aspect a) {aspects.add(a);}
    Msg execute(Msg m) {
        For each a in aspects do
            A.before()
        Try {
            Msg r = msg.execute();
            For each a in aspects do { a.onafter(r); }
        } catch (Exception e) {
            For each a in aspects do { a.onException(ex); }
        }
    }
}
```

- يُعرّف هنا صنف Facade الذي يُمثّل الواجهة الموحّدة لتنفيذ الرسائل. يحتفظ بقائمة aspects يُسجّل فيها كل الجوانب عبر الدالة register(). عند استدعاء execute()، يُنفّذ أولاً دالة before() لكل الجوانب، ثم يُنفّذ العملية الأساسية ضمن try؛ فإن نجحت تُستدعى دوال after()، وإن فشلت (استثناء) تُستدعى دوال onException() بدلاً منها. وبذلك يصبح إضافة سلوكٍ جديد (تتبع، أمان، معاملات) مستقلاً تماماً عن منطق العمل الأساسي.

- نلاحظ في الكود أنه لدينا تابع Register و هو مهم جداً لأنه عند يتم استدعاء الـ Aspect و يسمى IOC، و يمكن أن يسمى أيضاً Publish and Subscribe، حيث أن الـ FACADE لا يستدعي الـ Aspect ضمنه و إنما الـ Aspect يسجل نفسه عند الـ FACADE.

- من هنا يتم تطبيق مفهوم الـ Enterprise Architecture أي في حال إضافة Aspect جديدة لا يتم تعديل كود الـ FACADE و لكن ضمن الـ Aspect يقوم بتسجيل نفسه عند الـ FACADE عن طريق تابع الـ Register الموجود ضمن كود الـ FACADE.

```

Class Tracing implements Aspect {
    Msg before(Msg input) {print(msg);}
    Msg after(Msg result) {print(result);}
    Msg onException(Exception ex) {ex.printStackTrace();}
Static {
    // IoC design pattern or Hollywood principle
    Facade.register(new Tracing());
}
}

```

■ نلاحظ في الصورة استخدام IOC Design Pattern أو ما يسمى بـ Hollywood Principle، و هذا أول تطبيق عملي للواجهة: صنف Tracing (التتبع/التسجيل) يُطبّق Aspect. دالة before() تطبع الرسالة الواردة، و after() تطبع النتيجة، و onException() تطبع تتبع الخطأ. الكتلة static تُسجّل هذا الجانب تلقائياً في Facade عند تحميل الصنف — وهذا ما يُسمّى مبدأ هوليوود (Hollywood Principle)، إذ لا يستدعي الجانب الإطار، بل الإطار هو من يستدعيه.

ملاحظة مهمة :

في الـ Compiled Languages يتم تنفيذ الأكواد الموجودة ضمن الـ Main ، أما في الـ Interpreted Languages يتم تنفيذ الأكواد الموجودة ضمن Static {...} ، مثل الكود الموجود في الصورة.

ما الذي يختلف من تطبيق لآخر؟

طبقة الخدمات

ما أهمية الهيكل؟

تخيل شخص طلب منه بناء برج من المكعبات، في حال قام بوضعهم فوق بعضهم سيصل إلى حد معين بعدها سيقع البرج و يعود للبناء من البداية. أما في حال وضع كرتونة طويلة عندها سيستطيع سند البرج عليها و بناء برج أطول من السابق و حتى لو إنهار بعض البرج سيبقى نصفه ثابتاً.

سؤال مهم: من أهم الأمور التي يقوم بها البنيان المؤسساتي هي؟

فصل الخدمات عن بعضها البعض.

الـ Thread الواحد يستدعي جميع الأمور و الخدمات و الـ Aspect التي يحتاجها.



Message Design Pattern

تسمح بتحويل الاستدعاء التقليدي للتوابع باستعمال عمليات قفز jump إلى رسالة Message و هذا يسمح بـ:

- تحويل الاستدعاء إلى أمر قابل للمقاطعة.
- جعل الاستدعاء قابل للحفظ Serializable وبالتالي يمكنه عبور Middleware الشبكة عبر ولا يقتصر على المعالج نفسه.
- إمكانية التنصت والتسجيل.
- إضافة معاملات إضافية دون الحاجة لتغيير صيغة الاستدعاء.
- التحقق من صحة الاستدعاء at run time مما يسمح بالبرمجة التزايدية.
- توحيد interface لجميع الخدمات مما يعزز إمكانية إعادة الاستعمال.

ماذا يعني Thread Safe يحقق الـ ACID ؟

MVC

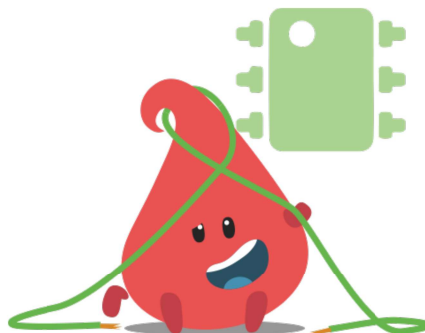
تهدف إلى فصل طبقة التخاطب مع الزبون client وهي طبقة view عن طبقة التعامل مع النموذج model عن التحكم Models و Views وتسمح بتعدد الـ controller.

مثال لتطبيق AOP

:AOP Business example / Before

```
// simple business logic: transfer from bank account to another
Boolean transfer(acc1, acc2, amount) {
    debit(acc1, amount);
    credit(acc2, amount);
}
```

- مثال بسيط على منطق العمل (Business Logic) قبل إضافة أي جوانب: دالة transfer() تُحوّل مبلغًا (amount) من حساب (acc1) إلى حساب آخر (acc2) عبر خطوتين فقط: debit() (سحب المبلغ من الحساب الأول)، ثم credit() (إضافته إلى الحساب الثاني). الشيفرة هنا نظيفة تمامًا ولا تحتوي على أي تتبع أو أمان أو معالجة أخطاء.



:AOP Business example + tracing / Before

```
// same with tracing
Boolean transfer( acc1, acc2, amount) {
    try {
        print("in"+ transfer+": "+acc1+", "+acc2+", "+ amount)
        debit(acc1, amount);
        credit(acc1, amount)
        print("out"+ transfer+": "+acc1+", "+acc2+", "+ amount)
    }
    Catch (exception e) {
        print("ex"+ transfer+": "+acc1+", "+acc2+", "+ amount+"\n"+message)
    }
}
```

- هنا تُدمج وظيفة التتبع (Tracing) يدوياً داخل الدالة نفسها: سطر يطبع بداية العملية ("in")، وسطر يطبع نهايتها بنجاح ("out")، وكتلة Catch تطبع تفاصيل الخطأ ("ex") عند الفشل. هذا يُظهر المشكلة الأساسية: شيفرة التتبع تختلط بمنطق العمل، فيصعب قراءتها أو إعادة استخدامها لاحقاً.

:AOP Business example + ACID / Before

```
// same with ACID transaction
Boolean transfer( acc1, acc2, amount) {
    try {
        print("in"+ transfer+": "+acc1+", "+acc2+", "+ amount)
        startTransaction();
        debit(acc1, amount);
        credit(acc1, amount)
        print("out"+ transfer+": "+acc1+", "+acc2+", "+ amount);
        commitTransaction();
    }
    Catch (exception e) {
        print("ex"+ transfer+": "+acc1+", "+acc2+", "+ amount+"\n"+message)
        rollback();
    }
}
```

- تُضاف الآن خاصية المعاملات (ACID Transaction) إلى الشيفرة السابقة: startTransaction() تبدأ المعاملة قبل التنفيذ، وcommitTransaction() تُثبتها بعد النجاح، أما rollback() فتراجع عن كل التغييرات عند حدوث خطأ. لاحظ أن كل وظيفة جديدة (تتبع، معاملات) تُضاف عبر تعديل الدالة نفسها، وهو أسلوب مكلف يزيد من تعقيد الشيفرة كل مرة.



:AOP Business example + Security / Before

```
Boolean transfer( acc1, acc2, amount) {
    If(!currentUser has right to transfer > amount ) throw exception;
    // Other security rules such as role and other amount values
    try {
        print("in"+ transfer+": "+acc1+", "+acc2+", "+ amount)
        startTransaction();
        debit(acc1, amount);
        credit(acc2, amount);
        print("out"+ transfer+": "+acc1+", "+acc2+", "+ amount);
        commitTransaction();
    }
    Catch (exception e) {
        print("ex"+ transfer+": "+acc1+", "+acc2+", "+ amount+"\n"+message)
        rollback();
    }
}
```

- تُضاف هنا أيضًا طبقة الأمان (Security) في بداية الدالة: يتحقق الشرط من أن المستخدم الحالي (currentUser) يملك الصلاحية الكافية لتحويل هذا المبلغ، وإلا تُرمى استثناء (exception). بهذا أصبحت دالة واحدة تحمل أربع مسؤوليات مختلفة في آن واحد: الأمان، التتبع، المعاملات، ومنطق العمل — وهذا بالضبط ما تسعى AOP إلى فصله.

:AOP Business example / After

```
Class ACID implements Aspect {
    Msg before(Msg input) {startTransaction();}
    Msg after(Msg result) {commitTransaction();}
    Msg onException(Exception ex){rollback()}
    Static {
        // IoC design pattern or Hollywood principle
        Facade.register(new ACID(), *);
    }
}
```

- صنّف جديد ACID يُطبّق واجهة Aspect ليحمل وظيفة المعاملات فقط: before() تبدأ المعاملة، و after() تُنهيها، و onException() تتراجع عنها عند الفشل. يُسجّل هذا الجانب في Facade عبر الرمز * الذي يعني تطبيقه على جميع العمليات في النظام.

:AOP Business example / After

```
Class Security implements Aspect {
    Msg before(Msg input) {checkUser();}
    Msg after(Msg result) {};
    Msg onException(Exception ex) {}
    Static {
        // IoC design pattern or Hollywood principle
        Facade.register(new Security(), *);
    }
}
```

- صنف آخر مستقل، Security، مسؤول فقط عن التحقق من صلاحية المستخدم عبر before() التي تستدعي checkUser(). أما after() و onException() فلا تقومان بأي إجراء هنا (تركنا فارغتين)، لأن وظيفة هذا الجانب تنتهي عند نقطة الفحص الأولى. يُسجّل بدوره في Facade على جميع العمليات (*) أيضاً.

بعد البنيان المؤسساتي:

```
Class BusinessSecurity implements Aspect {
    Msg before(Msg msg) {if(user does not have the right
    to transfer > msg.amount) throw exception ;}
    Msg after(Msg result) {;}
    Msg onException(Exception ex) {}
Static {
    // IoC design pattern or Hollywood principle
    Facade.register(new BusinessSecurity(), transfer);
}
```

- مثال أكثر تخصصاً: صنف BusinessSecurity يطبّق فحص أمان خاصاً بمنطق العمل نفسه، حيث تتحقّق before() من أن المستخدم يملك صلاحية تحويل مبلغ بهذا المقدار (msg.amount) وإلا تُرمى استثناء. الفرق الجوهرى عن الصفحة السابقة أن التسجيل في Facade يستخدم الكلمة transfer بدلاً من *، أي أن هذا الجانب يُطبّق فقط على عملية التحويل وليس على جميع العمليات.

:AOP Business example / After

```
// simple business logic: transfer from bank account to another
Boolean transfer (Msg msg) {
    debit ( msg.acc1, msg.amount );
    credit ( msg.acc2, msg.amount );
}
```

- هذه نتيجة تطبيق AOP: عادت دالة transfer() إلى بساطتها الأصلية، لكنها الآن تستقبل كائن رسالة واحداً (msg) يحمل بياناته (acc1, acc2, amount). لا أثر للتبع أو الأمان هنا، لأن Facade هو من يتولى تنفيذها قبل و بعد استدعاء هذه الدالة